Docker Shell

Version: 1.0.4
Release Date: 2026-06-07
Author: Glenn Madine
Language: Rust
Minimum Recommended Docker Version: 29.5.2 (build 79eb04c)

Overview

Docker Shell is a cross-platform interactive command-line shell that wraps the Docker CLI with a familiar set of file-system built-ins. It provides an ergonomic terminal experience for managing Docker projects — including directory navigation, file operations, and project scaffolding — without leaving the shell.

Any command not recognised as a built-in is forwarded directly to the `docker` binary as-is.

Features

- Interactive REPL with a prompt showing the current working directory
- Colour-coded output via the `colored` crate
- File and directory management built-ins (create, delete, rename, copy, list)
- Docker project scaffolding (`NEW`)
- Pass-through to Docker for all unrecognised commands
- Cross-platform: Windows, macOS, Linux
- UTF-8 and ANSI colour support on Windows 10+ via raw WinAPI FFI (no external winapi crate required)

Dependencies

Crate	Purpose
`colored`	Coloured terminal output

Crate	Purpose
`filetime`	Updating file modification timestamps (`TOUCH`)

Standard library modules used: `std::io`, `std::process`, `std::env`, `std::fs`, `std::path`, `std::time`.

Built-in Commands

Command	Description
`HELP`	Display the help screen
`CD <path>`	Change the current working directory
`COPY <source> <dest>`	Copy a file. If `dest` is an existing directory, copies into it. Creates parent directories as needed.
`CLEAR` / `CLS`	Clear the terminal screen using ANSI escape codes
`DEL <file>`	Delete a file (not directories — use `RMDIR` for those)
`DIR` / `LS [path]`	List directory contents. Defaults to current directory. Sorts directories first, then files, both alphabetically. Shows size and last-modified timestamp.
`DO <command>`	Execute an arbitrary system command
`MKDIR` / `MD <dir>`	Create a new directory (supports nested paths, e.g. `a/b/c`)
`NEW <project>`	Scaffold a new Docker project: creates a folder, empty `Dockerfile`, and `.env`, opens it in the system file manager, and `cd`s into it
`REN <old> <new>`	Rename a file or directory
`RMDIR` / `RD [-r] <dir>`	Remove a directory. Use `-r` / `--recursive` to remove non-empty directories
`TOUCH <file>`	Create an empty file, or update its modification timestamp if it already exists
`VER`	Print the current shell version

Command	Description
`EXIT`	Quit Docker Shell
(anything else)	Forwarded directly to `docker` as arguments

> **Note:** Prefixing a command with `docker` (e.g. `docker ps`) works but is unnecessary and will trigger a reminder message.

Architecture

Entry Point — `main()`

1. Calls `setup_console()` to configure the terminal (UTF-8 + ANSI on Windows).
2. Sets the window title to `Docker Shell <version>` via an ANSI escape sequence.
3. Clears the screen and prints the startup banner.
4. Enters a **REPL loop**:
   - Prints a prompt: `✨ <current_dir> ▶️`
   - Reads a line from stdin; handles EOF gracefully.
   - Splits input into a `keyword` and `rest` on the first whitespace.
   - Dispatches to the appropriate handler via `match keyword.to_uppercase()`.
   - Unrecognised keywords are passed to `run_docker()`.

Function Reference

`main()`

Program entry point. Initialises the terminal, displays the banner, and runs the REPL loop.

`setup_console()`

Platform-specific.

- **Windows:** Enables UTF-8 output (code page 65001) and ANSI virtual terminal processing via raw WinAPI FFI calls (`SetConsoleOutputCP`, `GetStdHandle`, `GetConsoleMode`, `SetConsoleMode`). No external winapi crate is required.

- **macOS / Linux:** No-op — UTF-8 and ANSI are supported by default.

`get_current_dir() -> Result<String, String>`

Returns the current working directory as a `String`, or an error message string on failure.

`run_docker(args_str: &str)`

Splits `args_str` on whitespace and executes `docker <args>`. Prints an error if Docker exits with a non-zero status or cannot be found.

`run_shell_command(cmd_line: &str)`

Implements the `DO` built-in. Splits `cmd_line` into a program name and arguments, then spawns the process and waits for it to finish.

`change_directory(path: &str) -> io::Result<()>`

Wraps `env::set_current_dir()` to change the process's working directory. Affects the prompt on the next iteration.

`list_directory(path: &str) -> Result<(), Box<dyn std::error::Error>>`

Implements `DIR` / `LS`. Reads directory entries, sorts them (directories before files, both alphabetically), and prints each entry with:

- An emoji icon (`📁` for directories, `📄` for files)
- File size in bytes (files only)
- Last-modified timestamp (UTC) via `format_modified()`

`format_modified(time: SystemTime) -> String`

Converts a `SystemTime` to a UTC timestamp string in the format `YYYY-MM-DD HH:MM:SS UTC`. Uses

only the standard library — no `chrono` or similar crate. Delegates calendar conversion to `days_to_ymd()`.

`days_to_ymd(days: u64) -> (u64, u64, u64)`

Converts a count of days since the Unix epoch (1970-01-01) into a `(year, month, day)` tuple. Accounts for leap years via `is_leap()`.

`is_leap(y: u64) -> bool`

Returns `true` if year `y` is a leap year using the standard Gregorian rule:
divisible by 4, except centuries unless also divisible by 400.

`handle_touch(rest: &str)`

Implements `TOUCH`. If the file exists, updates its modification timestamp using `filetime::set_file_mtime()`. If it does not exist, creates an empty file via `OpenOptions`.

`handle_new_project(name: &str)`

Implements `NEW`. Steps:

1. Creates the project directory with `fs::create_dir_all()`.
2. Creates an empty `Dockerfile` inside it.
3. Creates an empty `.env` inside it.
4. Opens the folder in the native file manager via `open_folder_in_explorer()`.
5. Changes the working directory into the new project folder.

`open_folder_in_explorer(path: &str)`

Opens a folder in the native file manager using platform-specific commands:

- **Windows:** `explorer <path>`
- **macOS:** `open <path>`

- **Linux:** `xdg-open <path>`

The process is spawned without waiting (`.spawn()` not `.status()`), so the shell remains responsive immediately.

`handle_mkdir(rest: &str)`

Implements `MKDIR` / `MD`. Creates a directory (and any missing parents) using `fs::create_dir_all()`. Reports an error if the path already exists.

`handle_rmdir(rest: &str)`

Implements `RMDIR` / `RD`. Accepts an optional `-r` / `--recursive` flag:

- Without `-r`: calls `fs::remove_dir()` — fails if the directory is non-empty, with a helpful hint to use `-r`.
- With `-r`: calls `fs::remove_dir_all()`.

`handle_copy(rest: &str)`

Implements `COPY`. Expects exactly two arguments: source and destination.

- If the destination is an existing directory, the file is copied into it, preserving the source filename.
- Creates any missing parent directories in the destination path.
- Reports the number of bytes copied on success.
- Refuses to copy directories (use other tooling for that).

`handle_delete(rest: &str)`

Implements `DEL`. Deletes a single file using `fs::remove_file()`. Refuses to delete directories and suggests `RMDIR` instead.

`handle_clear()`

Clears the terminal by writing the ANSI escape sequence `\x1B[2J\x1B[H` (erase screen + move cursor to home) and flushing stdout.

`handle_ren(rest: &str)`

Implements `REN`. Expects exactly two arguments: old name and new name. Uses `fs::rename()`. Errors if the source does not exist or the destination already exists.

Error Handling

All functions report errors to stderr via `eprintln!`. The REPL loop continues after non-fatal errors. Fatal I/O errors on stdin (e.g. broken pipe) cause a clean exit with a message. Docker and shell commands that exit with a non-zero status are reported but do not crash the shell.

Platform Notes

Platform	File Manager Command	Console Init
Windows	`explorer`	WinAPI FFI
macOS	`open`	None (default)
Linux	`xdg-open`	None (default)

Unsupported platforms will see a "Cannot open folder: unsupported platform." message for `NEW` but otherwise function normally.